Inference Engines

Benchmark https://github.com/sihyeong/Awesome-LLM-Inference-Engine

Tier 1: Production Server Engines (high QPS, data centre)

1. **vLLM** — github.com/vllm-project/vllm
   The default open-source production engine. Invented PagedAttention. Best flexibility, broad model support, easiest setup of the high-performance tier. Strong default engine when you need general LLM serving with good throughput, good TTFT, and hardware flexibility. Slight performance gap vs SGLang/LMDeploy now showing.
2. **SGLang** — github.com/sgl-project/sglang
   SGLang and LMDeploy achieve ~16,200 tok/s vs vLLM's ~12,500 tok/s i.e. a 29% gap even when vLLM uses the same FlashInfer kernels. The difference is RadixAttention (better prefix caching via radix tree) and tighter CPU scheduler design. Best choice for RAG, agents, and multi-turn chat.
3. **TensorRT-LLM** — github.com/NVIDIA/TensorRT-LLM
   NVIDIA's compilation-based engine. On B200 GPUs, TensorRT-LLM consistently outperforms both SGLang and vLLM across all metrics, thanks to its deeper optimisation for NVIDIA's latest hardware architecture. Highest peak performance on NVIDIA hardware; compile-time investment is significant. Requires Docker, fits naturally into the NVIDIA ecosystem (Triton, NIM, NeMo).
4. **LMDeploy** — github.com/InternLM/lmdeploy
   From the InternLM team at Shanghai AI Lab. Pure C++ TurboMind backend eliminates Python overhead entirely, achieving throughput equivalent to SGLang. Best quantisation support for 4-bit models. Strong choice for NVIDIA-centric deployments that want max throughput with less setup complexity than TRT-LLM.
5. **Nvidia Dynamo** — github.com/triton-inference-server/server
   A general-purpose inference platform supporting PyTorch, TensorFlow, ONNX, TensorRT, and multiple model types — not LLM-specific. Used as a wrapper around TRT-LLM for multi-model, multi-framework serving at enterprise scale.
6. **HuggingFace TGI (Text Generation Inference)** — github.com/huggingface/text-generation-inference
   TGI entered maintenance mode in December 2025 but its still good for HF inferencing. Hugging Face now recommends vLLM or SGLang for new deployments but it still works in existing deployments. Was notable for deep HF ecosystem integration and strong prefix caching on long chat histories.

Tier 2: Research / Specialised Server Engines

1. **DeepSpeed-MII / DeepSpeed-FastGen** — github.com/microsoft/DeepSpeed-MII
   Microsoft's inference layer on top of DeepSpeed. Introduced Dynamic SplitFuse

(interleaves prefill chunks with decode steps, similar to chunked prefill). Good for very large models across many GPUs. Less actively developed for inference since 2024, as the team shifted focus to training.

2. **LightLLM** — github.com/ModelTC/lightllm
   Lightweight pure Python + Triton engine from ModelTC. TokenAttention (block size = 1 variant of PagedAttention). Efficient router for multi-instance serving. Fast to iterate on for research but less production-hardened than vLLM.
3. **Sarathi-Serve** — github.com/microsoft/sarathi-serve
   MSR India research engine. Key contribution: chunked prefill (piggybacking decode steps during prefill) — this idea was later adopted into vLLM 0.4+. Primarily a research artefact now.
4. **DistServe** — github.com/LLMServe/DistServe
   Research engine that disaggregates prefill and decode onto separate GPU pools, independently scaling each. Demonstrates 7.4× more requests at the same SLO vs baseline. Influential paper and the concept is being productionised in vLLM and SGLang.
5. **NanoFlow** — github.com/efeslab/NanoFlow
   Research engine focused on overlapping computation and communication via nano-batching — squeezes out idle GPU time between operations. Academic; not production-ready.
6. **PowerInfer** — github.com/SJTU-IPADS/PowerInfer
   CPU+GPU heterogeneous inference from SJTU. Precomputes "hot" neuron activations that stay on GPU; "cold" activations run on CPU. Designed for consumer hardware with limited VRAM.

Tier 3: Local / Edge / CPU Engines

1. **llama.cpp** — github.com/ggml-org/llama.cpp
   The universal local inference engine. Pure C/C++, no dependencies. Supports 1.5-bit through 8-bit quantisation, Apple Silicon via Metal, NVIDIA via CUDA, AMD via HIP, x86 via AVX2/AVX512. GGUF is its model format, now the standard for local models. 85,000+ GitHub stars. The foundation under Ollama, LM Studio, and many local AI tools.
2. **Ollama** — github.com/ollama/ollama
   A polished UX wrapper around llama.cpp. Two commands to run any model locally. OpenAI-compatible API out of the box. Not designed for high-concurrency production. It's the prototyping and developer tool. Only Ollama and LLaMA.cpp passed all stability tests across concurrency levels in benchmarks. Do not recommend. Very slow.
3. **KTransformers** — github.com/kvcache-ai/ktransformers
   CPU-GPU heterogeneous inference specialised for MoE models (DeepSeek, Mixtral). Supports DeepSeek-R1 and V3 on a single 24GB VRAM GPU with 382GB DRAM — up to 3–28× speedup vs llama.cpp. The answer for running 671B MoE models on consumer/workstation hardware. SOSP 2025 paper.
4. **ExLlamaV2** — github.com/turboderp/exllamav2
   Highly optimised consumer GPU inference. Custom EXL2 quantisation format

(Hessian-aware, better quality than GPTQ at same bit-width). Best tokens/sec on single consumer GPU (RTX 4090 etc). Popular in the local AI hobbyist community.

5. **MLC-LLM** — github.com/mlc-ai/mlc-llm
   Compiler-driven universal deployment engine from the TVM/Apache community. Uses ML compilation via Apache TVM to automatically generate portable GPU libraries for CUDA, Metal, Vulkan, and WebGPU. The only engine that natively targets iOS, Android, and browser (WebGPU) alongside servers. Good portability; weaker concurrency scaling than vLLM/SGLang.
6. **WebLLM** — github.com/mlc-ai/web-llm
   MLC-LLM's browser-side sibling. Runs LLMs entirely in-browser via WebGPU and no server required. NPM package. The only option for fully client-side, zero-server LLM inference.
7. **IPEX-LLM (Intel)** — github.com/intel/ipex-llm
   Intel's acceleration layer for their own hardware (Arc GPUs, Xeon CPUs, NPUs). Integrates with llama.cpp, Ollama, vLLM, HuggingFace, and DeepSpeed and essentially a hardware backend adaptor rather than a standalone engine. Relevant if you're on Intel infrastructure.
8. **MLX (Apple)** — github.com/ml-explore/mlx
   Apple's array framework for Apple Silicon. Not strictly an inference engine but the foundation for fast local inference on M-series Macs. mlx-lm wraps it for LLM use. Metal-native, unified memory is often faster than llama.cpp on Apple hardware for certain models.

Tier 4: Commercial / Managed (API-only)

1. **Groq** (groq.com) LPU hardware — deterministic sub-50ms latency, fixed model selection
2. **Together AI** OpenAI-compatible API, broad OSS model selection, speculative decoding
3. **Fireworks AI** Fastest OSS model API, FireAttention kernel, JSON mode
4. **Friendli Inference** Managed single-tenant, enterprise SLAs
5. **Cerebras** Wafer-scale chip, sub-30ms TTFT on 8B models
6. **SambaNova** Dataflow chips, good for large batch offline inference

And then there are more than 100 Neo-cloud companies that allow you to serve vlln/sglang etc.